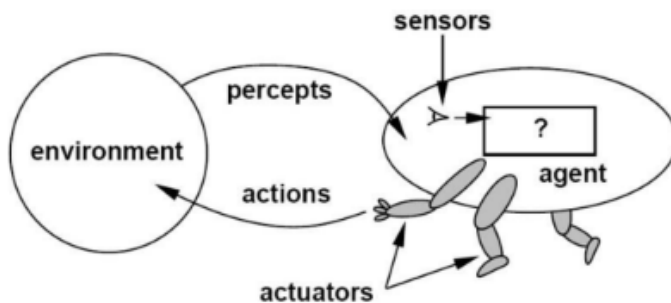


<https://drive.google.com/file/d/1ZgvNWRWusvPWmg8sYTONzABzDNYrA5vp/view>

★ Agents

- agent is anything that can be viewed as **perceiving its environment through sensors** and **acting upon that environment through actuators**.
- **Human agent:**
 - Eyes, ears, and other organs for sensors;
 - hands, legs, mouth, and other body parts for actuators
- **Robotic agent:**
 - Cameras and infrared range finders for sensors;
 - various motors for actuators.
- **Software agent:** Software agent is a **computer program** that **acts for a user or other program**: an **agreement to act on one's behalf**.

★ Agents and Environments



- agent function **maps from percept histories to actions: $[f: P^* \rightarrow A]$**
- **agent program** runs on the **physical architecture** to **produce f**
Agent = Architecture + Program

Vacuum Cleaner World

- **simple, two-room environment** used to model how an intelligent agent works.
- **Environment:**
 - Two locations: **A** and **B**
 - Each location can be:

- **Clean**
- **Dirty**

The vacuum cleaner (agent) is in one of the two rooms.

- 👁️ **Percepts (What the Agent Sees)**

The agent perceives: [Location, Status]

Example:

- [A, Dirty]
- [B, Clean]

So the agent only knows:

- Where it is
- Whether that room is dirty or clean

⚠️ **It does NOT know the condition of the other room.**

The pseudocode:

```
pgsql

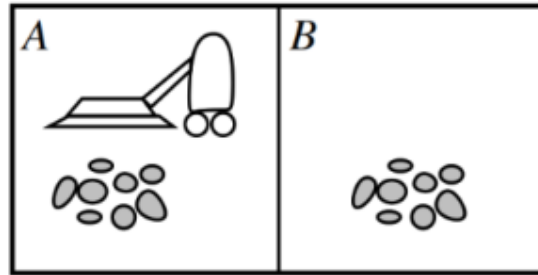
function REFLEX-VACUUM-AGENT(location, status):

    if status = Dirty then return Suck
    else if location = A then return Right
    else if location = B then return Left
```

- ⚙️ **Actions (What the Agent Can Do)**

The agent can perform 4 actions:

- **Left**
- **Right**
- **Suck**
- **DoNothing**



Tabulation of an agent function of the vacuum-cleaner

Percept sequence	Action
$[A, Clean]$	<i>Right</i>
$[A, Dirty]$	<i>Suck</i>
$[B, Clean]$	<i>Left</i>
$[B, Dirty]$	<i>Suck</i>
$[A, Clean], [A, Clean]$	<i>Right</i>
$[A, Clean], [A, Dirty]$	<i>Suck</i>
$\vdots$	$\vdots$

★ Rational Agents

- An agent should strive to "**do the right thing**", based on what it can perceive and the actions it can perform.
- The **right action is the one that will cause the agent to be most successful**.
- For each possible percept sequence, a rational agent should **select an action**
- **that is expected to maximize its performance measure**, given the **evidence provided by the percept sequence** and whatever **built-in knowledge** the agent has.
- **Performance measure:** **criterion** used **to evaluate how successful an agent is in achieving its goal**.
- Eg; performance measure of a vacuum-cleaner agent could be: amount of dirt cleaned up, amount of time taken, amount of electricity consumed, amount of noise generated, etc.
- An agent is **autonomous** if its **behavior is determined by its own experience** (with ability to learn and adapt).

★ PEAS

[Performance measure, environment, actuators, sensors]

- PEAS to be kept in mind when we design a rational/intelligent agent

Agent: automated taxi driver

- Performance measure: *Safe, fast, legal, comfortable trip, maximize profits*
- Environment: *Roads, other traffic, people and objects in/around the street*
- Actuators: *Steering wheel, accelerator, brake, signal, horn*
- Sensors: *Cameras, sonar, speedometer, GPS, odometer, engine sensors, keyboard*

Agent: Medical diagnosis system

- Performance measure: Healthy patient, minimize costs, lawsuits
- Environment: Patient, hospital, staff
- Actuators: Screen display (questions, tests, diagnoses, treatments, referrals)
- Sensors: Keyboard (entry of symptoms, findings, patient's answers)

Agent: Part-picking robot

- Performance measure: Percentage of parts in correct bins
- Environment: Conveyor belt with parts, bins
- Actuators: Jointed arm and hand
- Sensors: Camera, joint angle sensors

Agent: Interactive English tutor

- Performance measure: Maximize student's score on test
- Environment: Set of students
- Actuators: Screen display (exercises, suggestions, corrections)
- Sensors: Keyboard

★ Environment Types [12]

1) Fully observable

- Agent's **sensors can measure all relevant aspects of the environment** at each point in time.
- tictactoe

2) Partially observable

- cards

3) Deterministic

- **Next state** of environment is **completely determined by current state** and **action executed by agent**
- vaccum

4) Stochastic (non-deterministic)

- Taxi driver

5) Episodic

- The agent's experience is **divided into atomic "episodes"** (each episode consists of the agent perceiving and then performing a single action), and the choice of action in each episode depends only on the episode itself.
- **Action depends on the current episode only !!!**
- Each decision is **independent**, and the **current action does NOT affect future decisions.**
- No memory needed
- **Robot , spam email detection**
- **Each action independent**

6) Sequential (non-episodic)

- **Current actions affect future states and future decisions.**
- Memory needed + Long-term impact
- Chess, taxi driver

◆ Episodic vs Sequential Environment

Feature	Episodic Environment	Sequential Environment
Definition	Experience divided into independent episodes	Actions affect future states and decisions
Decision Basis	Action depends only on current percept	Action depends on current state + history
Dependency	Decisions are independent	Decisions are interdependent
Memory Requirement	No memory needed	Memory is required
Long-Term Impact	No long-term consequences	Long-term consequences exist
Planning Required	Not necessary	Necessary
Complexity	Simpler	More complex
Example	Spam email filtering, image classification	Chess, self-driving taxi, robot navigation

7) Static

- **environment** is **unchanged** while an agent is thinking.
- chess

8) Dynamic

- Env changes while agent thinks
- Taxi driver

[**Semi-dynamic:** env doesn't change but agent's performance score does, with time.
The longer it takes → less score]

9) Discrete

- **Limited & distinct percepts n actions**
- Chess has **finite** number of distinct states
- **Finite states and actions**

10) continuous

- **No limit.**
- Taxi driver

11) Single

- An agent operating by itself in an env
- crossword

12) Multiple

- Chess is multi-agent

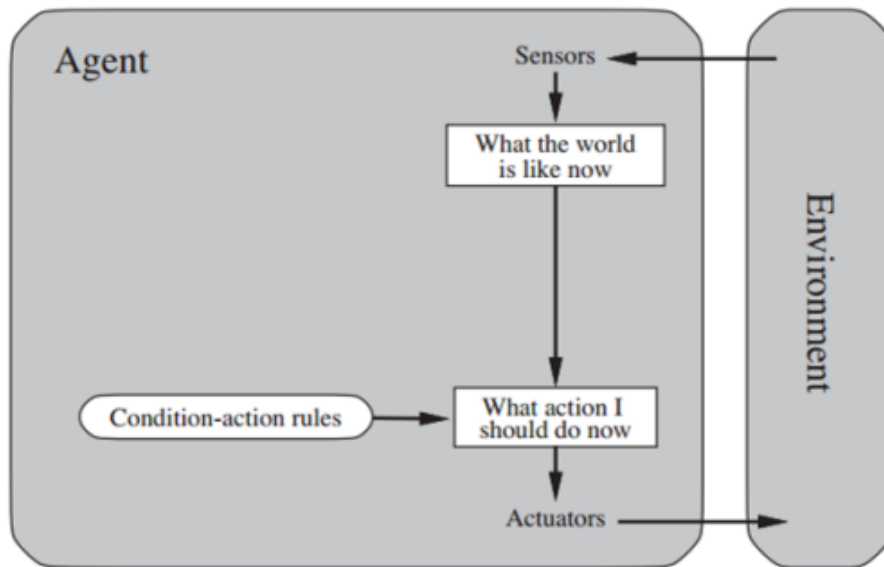
- The environment type largely determines the agent design
- The **real world** is (of course) partially observable, stochastic, sequential, dynamic, continuous, multi-agent.

★ Agent Types [05 types]

- Division on the basis of **perceived intelligence and capacity to change environment**
 - 1) **Simple reflex** agents
 - 2) **Model-based reflex** agents
 - 3) **Goal based** agents
 - 4) **Utility based** agents
 - 5) **Learning** agents

1) Simplex reflex agents

- Agent **doesn't know prev state, selects actions based on curr percept and condition**
- agent **selects an action(s)** based on the **current precept and conditions**, ignoring the rest of the precept history (previous state).
- **if x happens, do y**
- Very limited Intelligence
- No Knowledge/Perception about the previous or next state
- **Operates in Partially Observable Environments**
- **Infinite loops are unavoidable**
- Eg; alarm clock

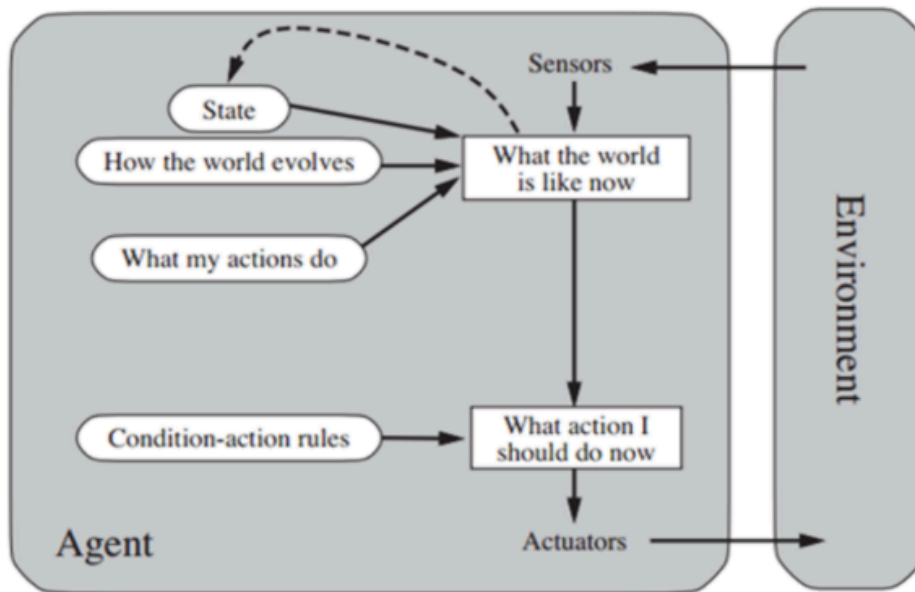


2) model based reflex agents

- agent decides its action(s) based on a **predefined set of condition-action rules**.
- State is evaluated in terms of **how it changed from the previous state**.
- Agent **keeps track of prev state** and **updates it using new percept**
- Eg; Roomba Cleaner Robot
 - > Remembers which areas are already cleaned
 - > Builds a map
 - > Avoids re-cleaning same space

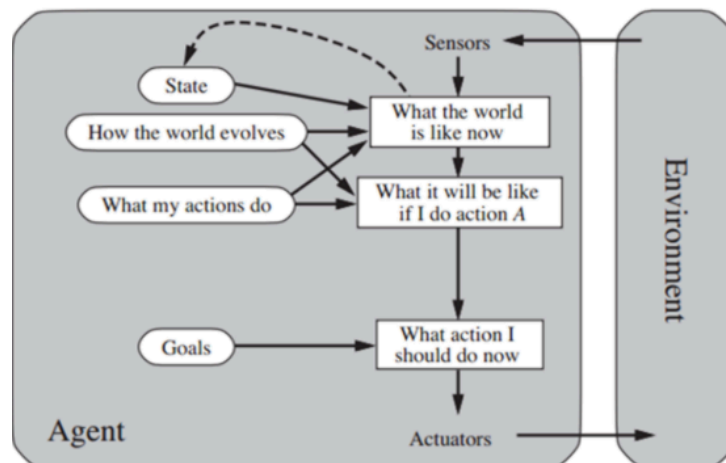
Structure of Model-Based Reflex Agent

1. Receive percept
2. Update internal state
3. Apply condition-action rules on state
4. Perform action



3) Goal-based Agents

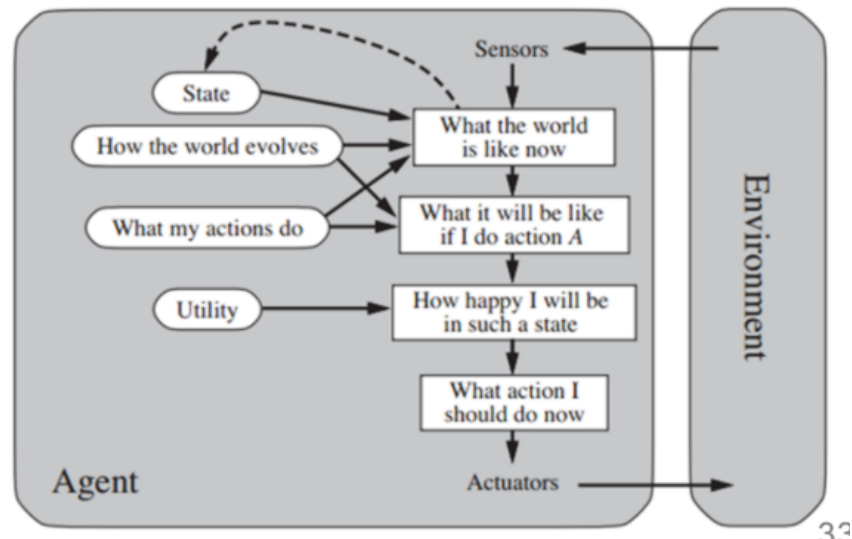
- agent decides its **action(s) based on a known goal.**
- Has **knowledge abt previous state**
- Involves **consideration of prev and future states**
- Selects actions by considering whether they help achieve a specific goal.
- Eg; GPS system finding a path to a certain destination



4) Utility-based Agents

- agent decides its action(s) **based on utilities/preferences.**

- **Optimality** [most favorable, or highest-performing outcome possible] is important
- **Chooses actions** that **maximize a utility function**
- Tries to achieve **best possible outcome**
- Eg; GPs finding shortest/fastest path



Goal-based agents only ask:

“Did I reach the goal?”

Utility-based agents ask:

“Among all possible goal states, which one is best?”

Because sometimes:

- Multiple solutions exist
- Some are better than others
- There is uncertainty

5) Learning Agents

- **Adapts** actions **based on feedback** (not just current percepts)
- Improves its performance over time by learning from experience and feedback.
- Can modify its own behaviour
- **Learns from experience/environment**

Learning element + performance element + critic + problem generator

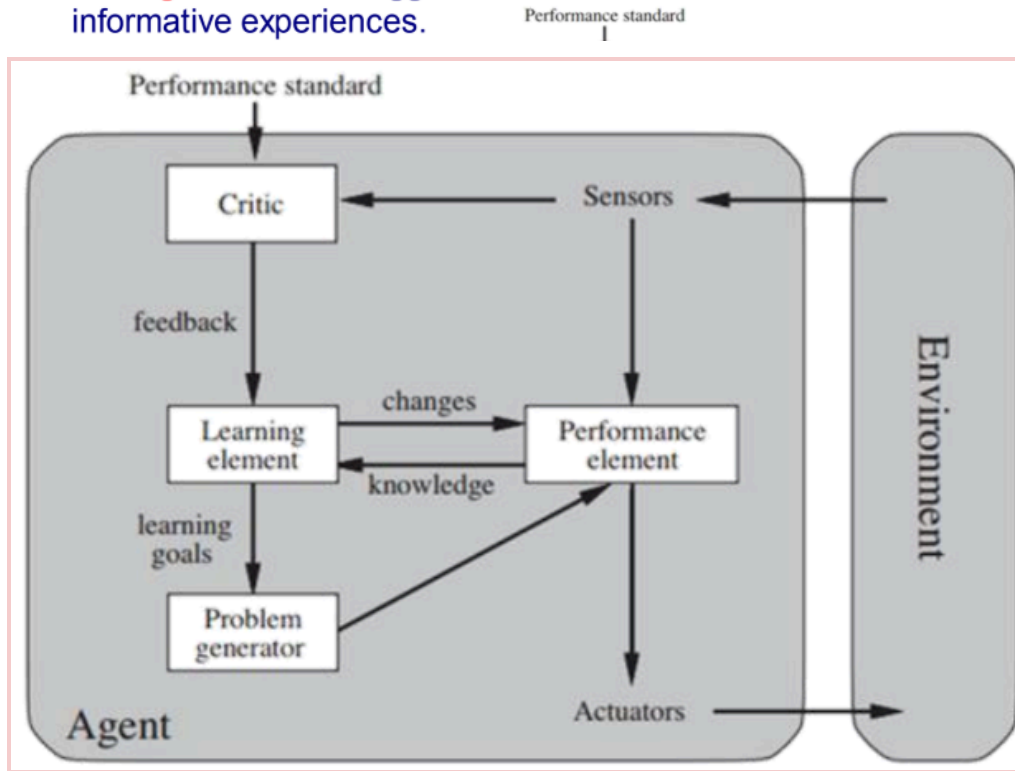
- Eg; human agent

Learning element - responsible for making improvements from past events.

Performance element - what we have up to now considered to be the entire agent. Takes percepts and then decides on actions.

Critic - gives the learning element feedback on how the performance element is doing and if it needs to be modified.

Problem generator - it suggests actions that lead to new and informative experiences.



1 Performance Element

- The part that chooses actions.
- Takes percepts from environment.
- Acts according to current knowledge.

👉 This is what we normally call "the agent."

2 Learning Element

- Improves the performance element.
- Uses past experiences.
- Updates internal knowledge/model.

👉 Responsible for making improvements.

3 Critic

- Evaluates how well the agent is performing.
- Compares performance with a standard.
- Provides feedback to learning element.

👉 Says: "You did well" or "You made a mistake."

4 Problem Generator

- Suggests exploratory actions.
- Encourages new experiences.
- Prevents agent from always choosing safest known action.

👉 Enables exploration.